PROJECT ATLAS

AI Tools & Their Usage

Technical overview for recruiter review

1. Project Overview

Project Atlas is a Report Copilot designed to help users ask questions about market-research reports and receive evidence-based answers. Its intended workflow combines document retrieval, language-model reasoning, calculations, source citations, conflict awareness, and escalation when an answer cannot be supported confidently. The application is built around a Python API and supporting data services.

2. AI Components and How They Are Used

AI tool / component	Purpose in Project Atlas
Large Language Model (LLM) API	Generates a natural-language answer from the question and evidence supplied by the application. The exact provider/model must be confirmed from project configuration. The assignment permits GPT, Claude, Gemini, or Llama, but that list alone does not prove which one this implementation uses.
Retrieval-Augmented Generation (RAG)	Combines document retrieval with language-model generation. Relevant report passages are retrieved and supplied as evidence, rather than relying only on the model's general knowledge.
Text embeddings	Represent report chunks and user questions as vectors so semantically related content can be retrieved even when wording differs. Confirm the exact embedding model from source/configuration.
Chroma vector store	Stores and searches vector representations of report chunks. Retrieval should use metadata and client-entitlement filters so unauthorized report content is not passed to the model.
Prompt design and instructions	Guide the model to answer from evidence, distinguish quoted facts from derived estimates, express uncertainty, and treat instructions found inside source documents as untrusted content.
Citation and evidence checks	The intended workflow links factual claims and numbers to report passages. Confirm that citations are validated in code; model-generated citations are not automatically reliable.
Deterministic calculation tool	Performs calculations such as CAGR using an explicit formula and assumptions, rather than relying on the model for arithmetic. Results not directly stated in a report should be labelled as derived estimates.
Conflict detection and escalation	When sources disagree or evidence is insufficient, the intended workflow should show the conflict or route the question for human review instead of silently selecting a value.

Prompt-injection and safety handling	Treats retrieved report text as data rather than instructions. Input validation, access checks, and read-only database restrictions are complementary safeguards; they are not supplied by the LLM alone.

3. Supporting Technologies (Not AI Models)

- **FastAPI:** Python API layer exposing application endpoints and connecting the interface to backend processing.
- **MySQL:** Relational storage for application data such as client/report metadata and, where implemented, escalation or conversation records.
- **Python:** Implements backend orchestration, retrieval, validation, and deterministic calculation logic.
- **PHP / CodeIgniter 3 integration:** The target legacy-portal integration specified by the assignment; describe it as implemented only if the relevant controller, library, and chat view are present and working.
- **Git and GitHub:** Version control and repository hosting; these are development tools, not AI tools.

4. Typical Request Flow

1. A user submits a question through the application.
2. The backend validates the request and identifies the requesting client.
3. The retrieval layer searches relevant report chunks and applies access filters before evidence is sent to the model.
4. The application may use deterministic calculation or a read-only database query for suitable questions.
5. The LLM drafts an evidence-grounded answer; the workflow is intended to attach citations and identify uncertainty or conflicts.
6. If evidence is insufficient or confidence is below the configured threshold, the request should be escalated rather than answered speculatively.

5. Responsible Use and Limitations

- A language model can produce incorrect statements; grounding, citation validation, and evaluation are essential.
- Access control must be enforced by the backend and retrieval layer, not merely requested in a prompt.
- Calculations should be reproducible and show formulas, source values, and assumptions.
- Conflicting sources should be shown with scope/date differences rather than averaged or silently resolved.
- Prompt-injection resistance requires layered controls and testing; a system prompt alone is not a security boundary.
- Do not describe a provider, model name, agent framework, monitoring product, or feature as implemented unless the repository/configuration confirms it.

6. Configuration Details to Confirm Before Submission

This document describes the AI architecture and intended responsibilities based on the Project Atlas assignment and available project context. The exact LLM provider/model, embedding model, agent framework, and status of each safety feature were not independently verified from the live source tree while preparing this document. Before sending it to a recruiter, confirm those names against code and environment-variable names (never include secret keys). If a component is planned rather than implemented, label it as planned.

7. Recruiter Summary

Project Atlas applies a retrieval-augmented generation approach to market-research question answering. It combines semantic search over report content with an LLM for natural-language responses, while using application-side logic for access filtering, calculations, evidence/citation handling, conflict awareness, and escalation. The goal is to make answers traceable to source material and prefer transparent uncertainty or human review over unsupported answers.